

FMCG Multi-Store Demand Forecasting

Role: ML Engineer | **Focus:** Classical ML + Deep Learning (Time-Series) | **Data:** data.csv

1. Business Context

You are an ML Engineer at a European FMCG retailer operating across 7 countries, 9 cities, and 13 stores (Hypermarket, Supermarket, Convenience, and E-commerce channels).

Leadership wants a demand forecasting system that:

- Predicts daily `units_sold` for each store × SKU combination
- Supports inventory planning to reduce stockouts and overstock
- Captures effects of promotions, price, weather, holidays, and channel

Historical data: ~1 million rows of daily sales, covering multiple SKUs across categories (Beverages, Dairy, Snacks, Personal Care, Home Care).

2. Dataset Overview

The provided file `data.csv` contains daily store–SKU observations with the following field groups. There are no missing values in the provided dataset.

Group	Fields
Time	<code>date</code> , <code>year</code> , <code>month</code> , <code>day</code> , <code>weekofyear</code> , <code>weekday</code> , <code>is_weekend</code> , <code>is_holiday</code>
Location	<code>store_id</code> , <code>country</code> , <code>city</code> , <code>channel</code> , <code>latitude</code> , <code>longitude</code>
Product	<code>sku_id</code> , <code>sku_name</code> , <code>category</code> , <code>subcategory</code> , <code>brand</code>
Demand / Sales	<code>units_sold</code> (primary target), <code>gross_sales</code> , <code>net_sales</code>
Pricing / Promo	<code>list_price</code> , <code>discount_pct</code> , <code>promo_flag</code> , <code>purchase_cost</code> , <code>margin_pct</code>
Inventory / Supply	<code>stock_on_hand</code> , <code>stock_out_flag</code> , <code>lead_time_days</code> , <code>supplier_id</code>
External	<code>temperature</code> , <code>rain_mm</code>

3. Problem Statement

Build an end-to-end forecasting pipeline that predicts the next 14 days of daily `units_sold` for every active (`store_id`, `sku_id`) pair.

Treat this as a production-oriented ML system, not a notebook demo. Data preparation, validation strategy, modeling, evaluation, and clear trade-off reasoning are all in scope.

4. Requirements

Part A — Data Understanding & Feature Engineering

- Perform exploratory analysis of seasonality, promo lift, stockouts, and store/SKU heterogeneity

- Create leakage-aware features only (no future information in training windows)
- Include these feature families:
 - Calendar / holiday / weekend features
 - Lag & rolling demand features (e.g., 7 / 14 / 28-day)
 - Price and promotion dynamics
 - Weather features
 - Store / category / channel encodings
- Explicitly handle stock_out_flag (censored demand vs. true demand)

Part B — Classical ML Baseline (Mandatory)

Train a baseline, for example:

- Tree-based model (LightGBM / XGBoost / CatBoost)

Report metrics and failure modes (sparse SKUs, promo spikes, cold starts).

Part C — Deep Learning Model (Mandatory)

Implement one sequence model that uses recent history per series, for example:

- LSTM / GRU encoder–decoder, or
- Temporal Convolutional Network (TCN), or
- Transformer / Temporal Fusion Transformer–style architecture

The Deep Learning solution must include:

- Multi-series modeling (shared weights across store–SKU series)
- Static covariates (store / channel / category) plus known future inputs (promo, calendar, and weather if assumed known)
- A train/validation strategy suitable for time series (no random row shuffle)

Part D — Evaluation & Decisioning

Suggested temporal split for Train, Test and Validation.

Metrics (report all):

- WAPE / MAE / RMSE overall
- Breakdown by channel, category, and promo vs. non-promo
- Optional business proxy: stockout risk or overstock cost using purchase_cost / margin_pct

Compare classical ML vs. Deep Learning: when each wins, and what you would ship first in production.

Part E — Engineering Judgment

Be ready to discuss:

- Scaling to more stores and SKUs
- Online vs. batch inference design
- Monitoring for drift and promo regime change
- What you would not put in a v1 release

5. Deliverables

Deliverable	Expectation
Working code	Reproducible pipeline on data.csv
Short write-up (1–2 pages)	Approach, assumptions, results, and next steps
Model comparison table	Baseline vs. Deep Learning on agreed metrics
30–45 min deep-dive	Architecture choices, leakage controls, and failure cases

6. Constraints

- Use only data.csv (no external datasets unless declared and justified)
- Prefer PyTorch or TensorFlow/Keras for the Deep Learning component
- Forecast horizon is fixed at 14 days
- Primary target is units_sold (not revenue), unless a multi-task setup is justified
- Do not use test-period labels during training or feature creation